

第 01 讲 - 嵌入式系统设计概述

一、应用举例

工业控制：工控设备、智能仪表、汽车电子

网络：网络设备、电子商务

军事国防：军事电子

消费电子：信息家电、智能玩具、通信设备、移动存贮

二、特点

经济性 小型化 可靠性 稳定性 高速度 实时性 智能性 实用性 交互性

三、发展趋势

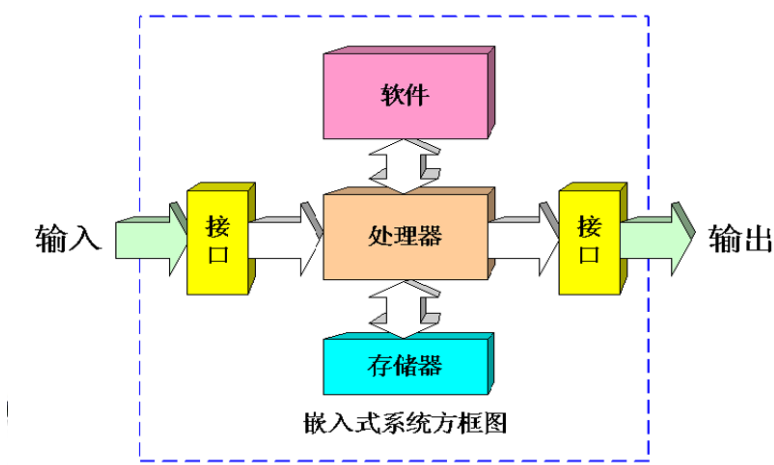
支持开发的工具和操作系统

联网成为必然趋势

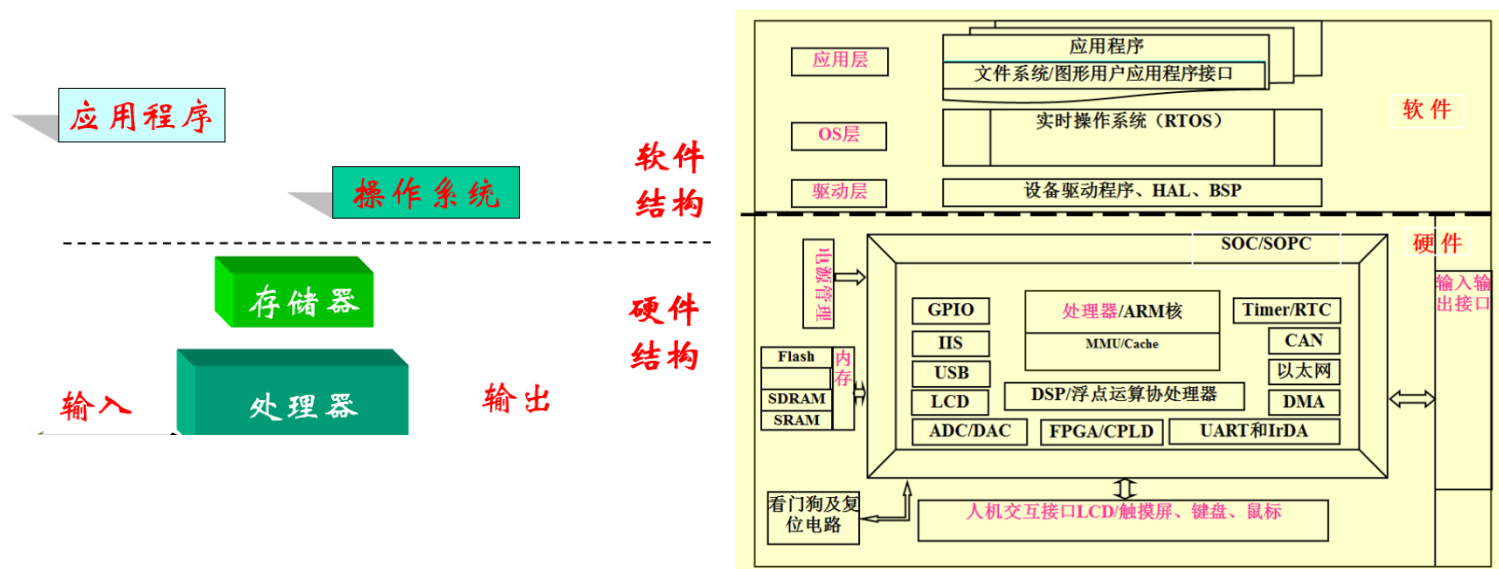
精简系统内核、算法，设备实现小尺寸、微功耗和低成本

提供精巧的多媒体人机界面

四、组成



五、架构



六、 硬件与软件要素

硬件

嵌入式处理器、各种类型存储器、模拟电路及电源、接口控制器及接插件

软件

操作系统 (OS)、板级支持包 (BSP)、设备驱动 (Device Driver)、协议栈 (Protocol Stack)、应用程序 (Application)

七、 嵌入式系统设计分析

详细分析嵌入式系统的需求

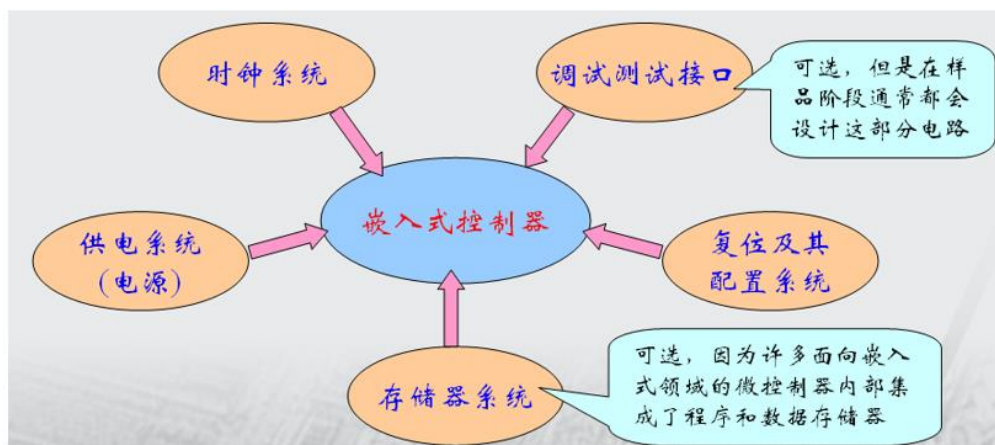
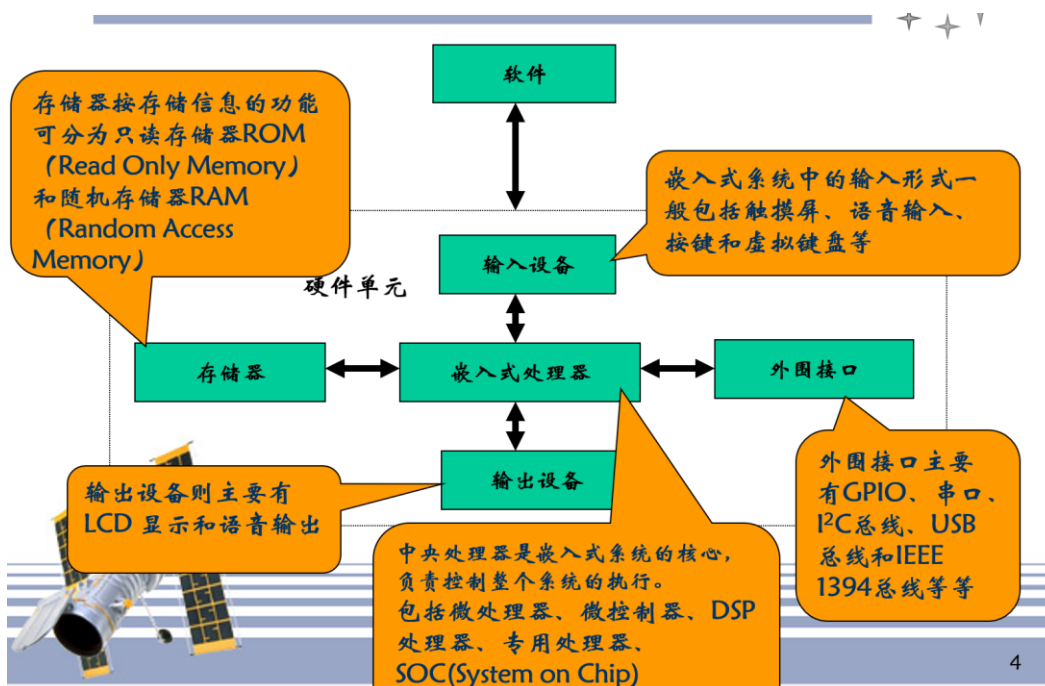
从需求引出系统设计思路

给出硬件设计和软件设计方法

结构设计、接口设计、程序设计（驱动设计、软件设计等）

第二讲 嵌入式处理器结构及技术

嵌入式系统硬件组成



存储系统对于最小嵌入式硬件

系统可以没有

嵌入式微处理器的集成度

- 用于桌面和服务器的微处理器的芯片内部通常只包括CPU核心、Cache、MMU、总线接口等部分，其他附加的功能如外部接口、系统总线、外部总线和外部设备独立在其他芯片和电路内。
- 嵌入式微处理器除了集成CPU核心、Cache、MMU、总线等部分外，还集成了各种外部接口和设备，如中断控制器、DMA、定时器、UART等。(符合嵌入式系统的低成本和低功耗需求，一块单一的集成了大多数需要的功能块的芯片价格更低，功耗更少)
- 嵌入式微处理器是面向应用的，其片内所包含的组件的数目和种类是由它的市场定位决定的。

在最普通的情况下，嵌入式微处理器包括：

片内存储器：部分嵌入式微处理器

LCD 控制器：面向终端类应用的嵌入式微处理器

定时器，A/D、D/A 转换器

总线

外部存储器的控制器，外设接口(串口，I2C)

中断控制器，DMA 控制器，协处理器

多媒体加速器：当高级图形功能需要时

其他标准接口或外设

嵌入式外围接口电路和设备接口

嵌入式外围设备：在嵌入式系统硬件构成中，除核心控制部件以外的各种存储器、输入/输出接口、作为人机接口的显示器/键盘、串行通信接口等。

根据外围设备的功能,可分为以下 5 类:

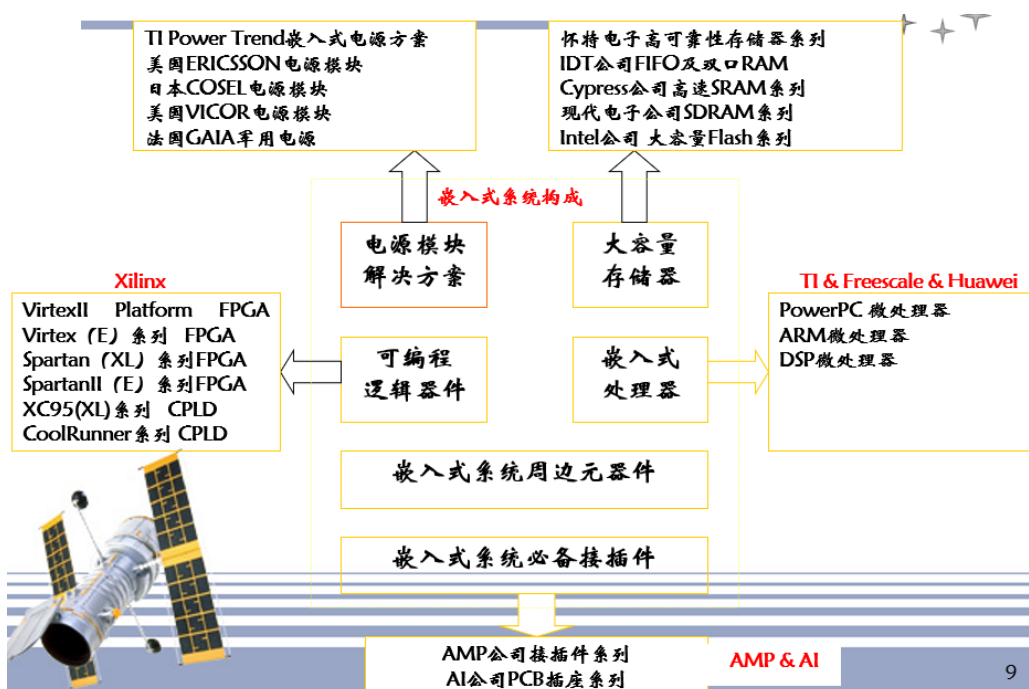
存储器

通信接口

输入输出设备

设备扩展接口

电源及辅助设备



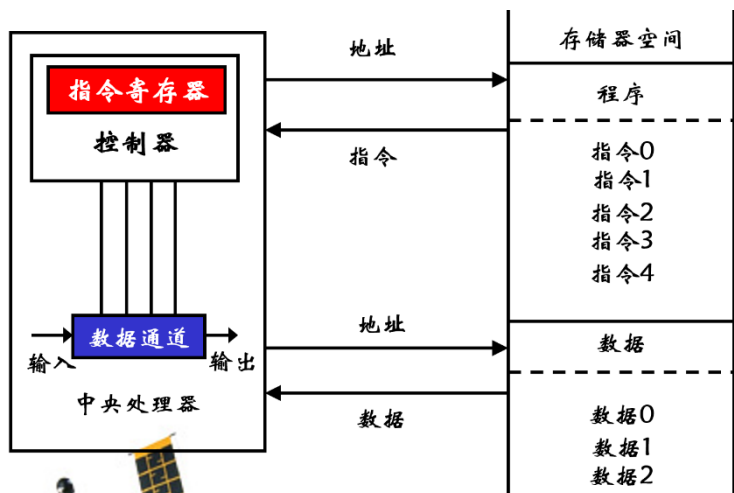
嵌入式处理器和硬件系统关键

冯·诺依曼体系结构模型

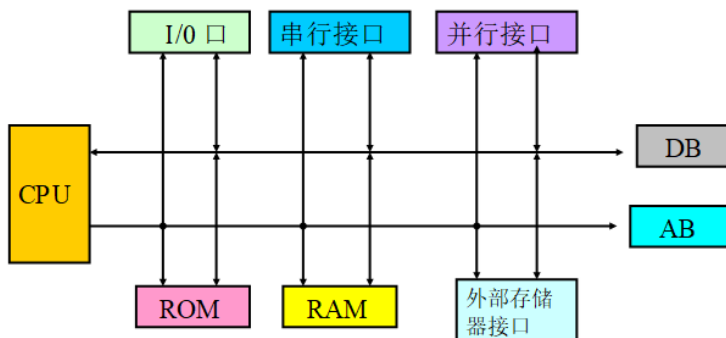
冯·诺依曼体系的特点：

- 数据与指令都存储在同一存储空间中
- 统一编址，指令和数据通过同一总线访问

- 被大多数计算机所采用
- 缺点：取指令和存取数据要从同一个存储空间存取，经由同一总线传输，因而它们无法重叠执行



冯·诺依曼结构示意图



哈佛体系结构

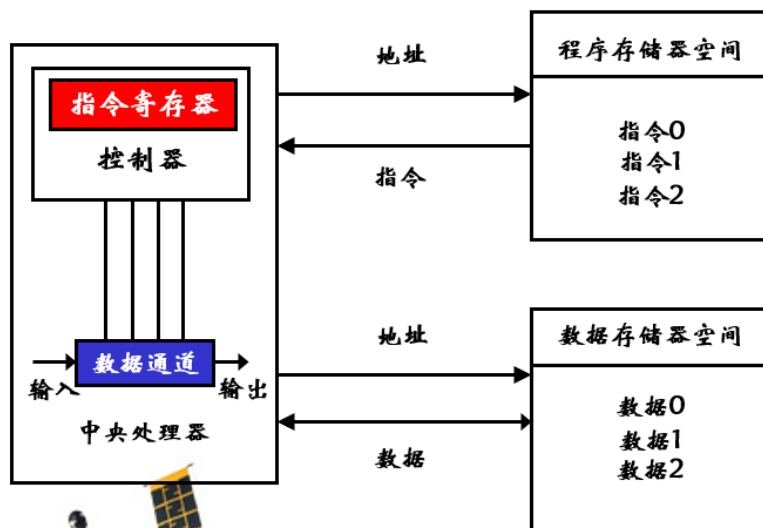
哈佛体系结构的特点：

- 程序与数据在不同存储空间；每个存储器独立编址、独立访问
- 使用程序和数据总线，使数据吞吐率提高一倍
- 适合于数字信号处理

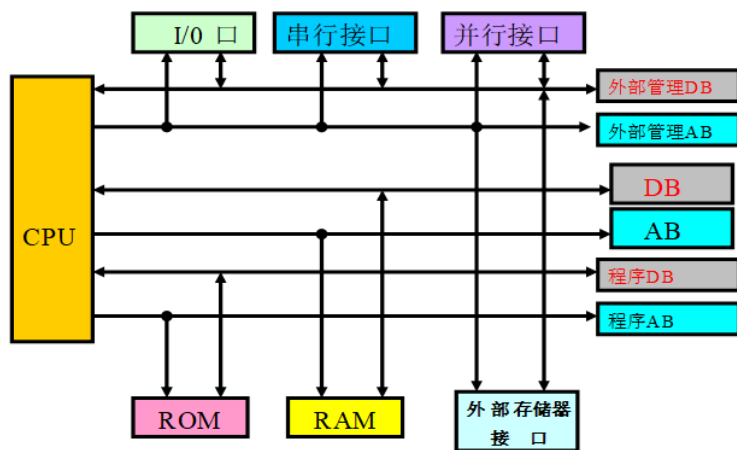
大多数 DSP 都是哈佛结构

由于取指令和存取数据分别经由不同的存储空间和不同的总线，使得各条指令可以重叠执行，这样，也就克服了数据流传输的瓶颈，提高了运算速度。

1944 年，哈佛大学的 Howard Aiken 发明了 Harvard Mark 系列计算机，并且在其中采用了分开的指令存储器和数据存储器，这就是后来著名的“哈佛结构”



哈佛体系结构示意图2



冯·诺依曼体系结构 vs. 哈佛体系结构

哈佛结构中数据存储器与程序存储器分开，需要 CPU 提供大量的数据总线，因而很少使用哈佛结构作为 CPU 外部构架

对于 CPU 内部，通过使用不同的数据和指令 Cache，可以有效的提高指令执行的效率。



因而目前大部分计算机体系都是在 CPU 内部的使用哈佛结构，在 CPU 外部使用冯·诺依曼结构。

CISC/RISC

CISC：复杂指令集（Complex Instruction Set Computer）

具有大量的指令和寻址方式

8/2 原则：80%的程序只使用 20%的指令———大多数程序只使用少量的指令就能够运行

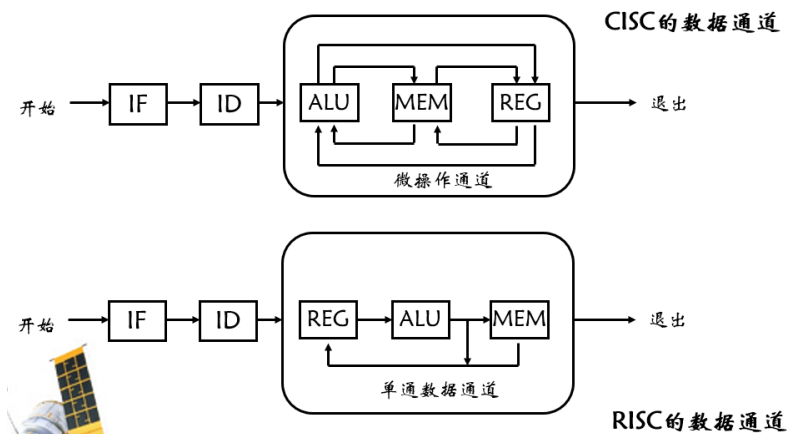
传统的 CISC 指令集随着计算机的发展引入了各种各样的复杂指令，使得指令集和为此要实现这些指令的计算机体系结构越来越复杂，已经不堪重负。

RISC：精简指令集（Reduced Instruction Set Computer）

在通道中只包含最有用的指令

确保数据通道快速执行每一条指令、使 CPU 硬件结构设计变得更为简单

- 指令规整、对称、简单，指令小于 100 条，基本寻址方式只有 2~3 种
- 单周期指令，便于流水操作
- 大量的寄存器（不少于 32 个）



	RISC	CISC
指令系统	由常用指令组成，使其简单高效。对不常用功能采用 组合指令 来实现。实现特殊功能采用 流水线 和 超标量 技术来弥补效率的劣势。	指令系统丰富，有专用指令来完成特定功能，处理特殊任务效率高
存储器操作	对存储器操作有限制，对控制简单化	存储器操作指令多，操作直接
程序	汇编程序需要较大的内存空间，实现特殊功能时程序复杂	汇编程序相对简单，科学计算及复杂操作的程序设计容易，效率较高
中断	在一条指令执行的适当地方可以中断	只能在一条执行结束后响应中断

CPU	包含较少的单元电路，面积小、功耗低	包含丰富的电路单元，功能强、面积大、功耗高
设计周期	结构简单，布局紧凑，设计周期短，易于采用新技术	结构复杂，设计周期长
用户使用	结构简单，指令规整，性能容易把握，易学易用	结构复杂，功能强大，实现特殊功能容易
应用范围	由于指令系统的确定与特定领域相关，故更适合于专用机	更适合通用机



RISC 和 CISC 的界限开始模糊

尽管 RISC 架构有不少优点，但决不能认为 RISC 架构就可以取代 CISC 架构，事实上，RISC 和 CISC 各有优势，而且界限并不那么明显。

现代的 CPU 往往采用 CISC 的外围，内部加入了 RISC 的特性，如超长指令集（VLSW）CPU 就是融合了 RISC 和 CISC 的优势，成为未来的 CPU 发展方向之一。

流水线

流水线技术：几个指令可以并行执行（使指令的执行速度或数据的处理速度更快，克服早期计算机指令执行和数据处理串行进行产生的处理器闲置时间的弊端）

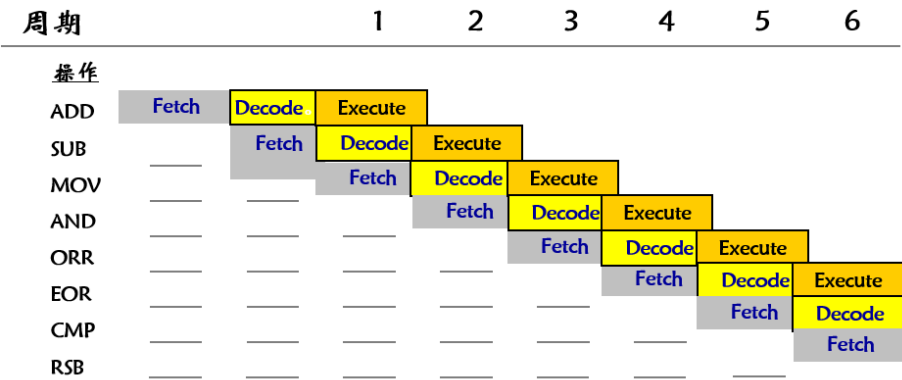
流水线：在 CPU 中把一条指令分解成多个可单独处理的操作，使每个操作在一个专门的硬件站（stage）上执行，这样一条指令需要顺序地经过流水线中多个站的处理才能完成，但是前后相连的几条指令可以依次流入流水线中，在多个站间重叠执行，因此可以实现指令的并行处理。

流水线能够正常工作的条件：任意时刻，每一级所使用的硬件必须能独立操作，不能多级同时占用同一硬件资源。

在正常情况下，每条指令都被划分成这样 3 个时钟周期来完成，即指令执行时间（Latency）是 3 周期（取指、译码，执行）。

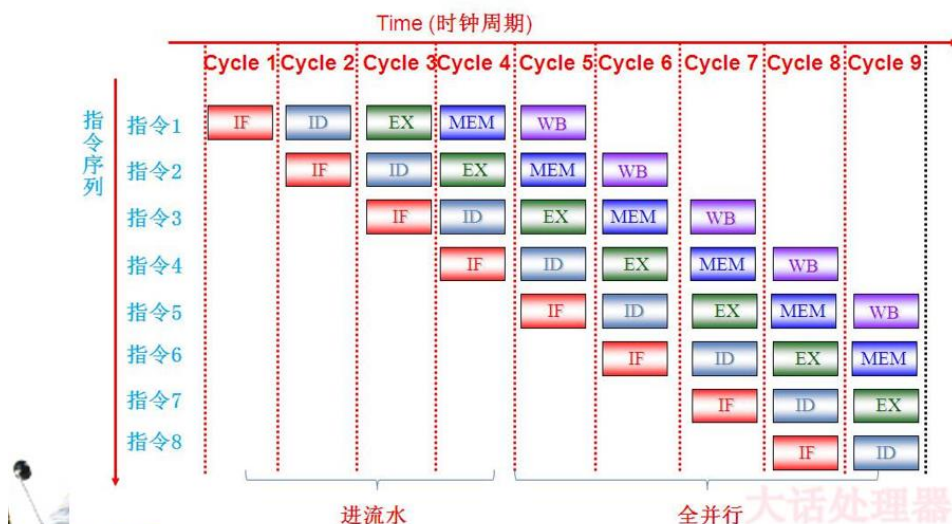
流水线的执行使得程序计数器 PC 必须在当前指令取指前计数。对于 ARM 处理器的 3 级流水线，以当前 PC 取指后，PC 值会增加为 PC + 4。

PC 指向正被取指的指令，而非正在执行的指令。



- 该例中用6个时钟周期执行了6条指令。
- 所有的操作都在寄存器中（单周期执行）。
- 指令周期数 (CPI) = 1。

史上最经典的5级流水线

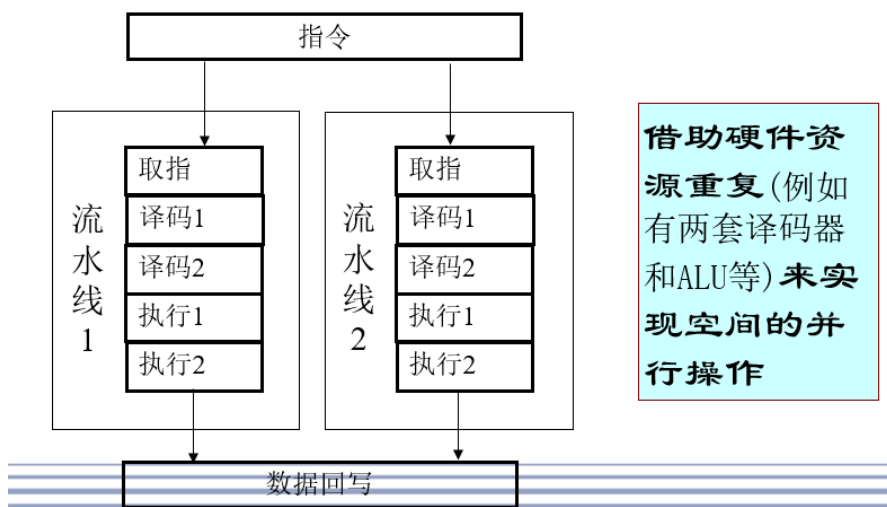


早期的MIPS、ARM9等处理器使用这种流水线，以后的处理器也都能看到它的影子，包括x86处理器等。

超标量及并行

超标量：配置多个执行部件和指令译码电路，能同时执行多条指令。

超标量执行：超标量 CPU 采用多条流水线结构。

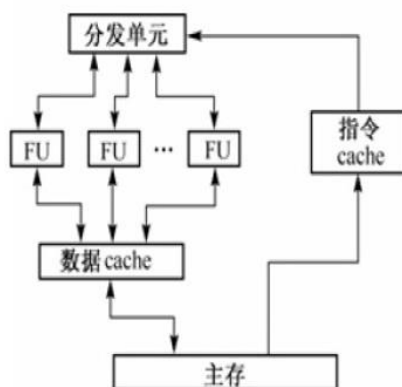


• 超标量处理器的结构

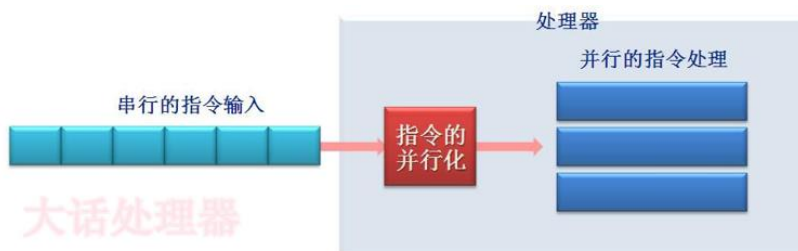
- 超标量机器的一个最突出的标志是处理器中有两个或两个以上的功能部件。为使多个功能部件能并行工作，其操作数之间必须没有相关性。

• 工作过程

- 取指令部件从指令cache中取出多条指令，并送至分发部件的指令缓冲器中；
- 每一个机器周期，分发部件都对指令窗口进行扫描，一旦发现可以并行发送的指令，并且与这些指令相应的功能部件是空闲的，则同时将它们送到功能部件去处理。

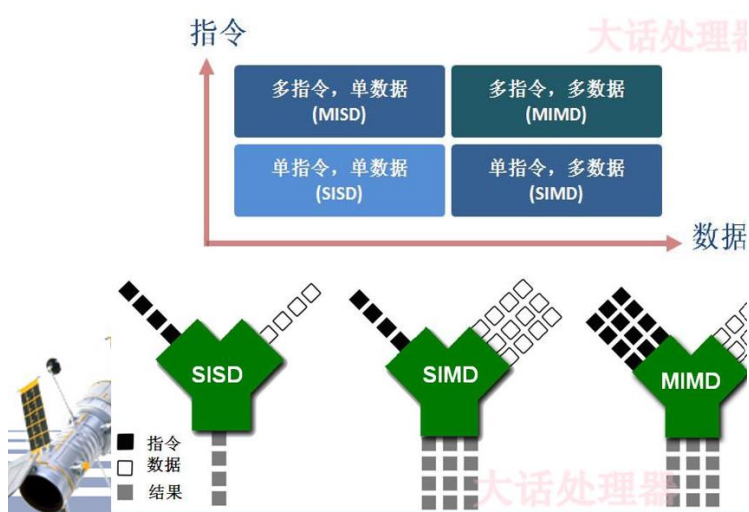
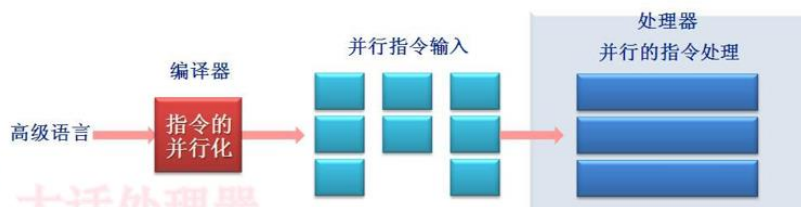


Superscalar 超标量



VLIIW 超长指令字

编译器的并行设计



并行总结



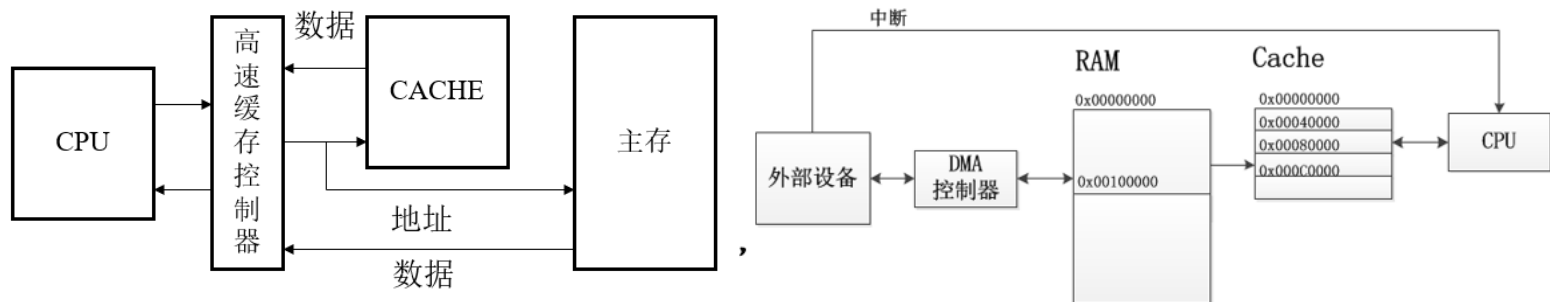
Cache (高速缓存)

1.Cache 的设计思想：把在一段时间内、一定地址范围中被频繁访问的信息集合，成批地从主存中读到一个能高速存取的小容量存储器中存放起来，供程序在这段时间内随时使用，从而减少或不再去访问速度较慢的主存，以加快程序的运行速度。

2.高速缓存的工作原理：

依据：程序和数据访问的局部性原理。

高速缓存是一种小型、快速的存储器，它保存部分主存内容的拷贝。



CACHE 刷新与回写

- 以 CPU 为核心看，将 **CPU 直接参与** 的事件称为**同步事件**，**CPU 未直接参与** 的称为**异步事件**。
- **Cache 的操作都是同步的**，但是如果在写一个外部设备的驱动时为了减少 CPU 的参与，用 DMA 来搬移数据，那么 **DMA 搬移数据是异步事件**。

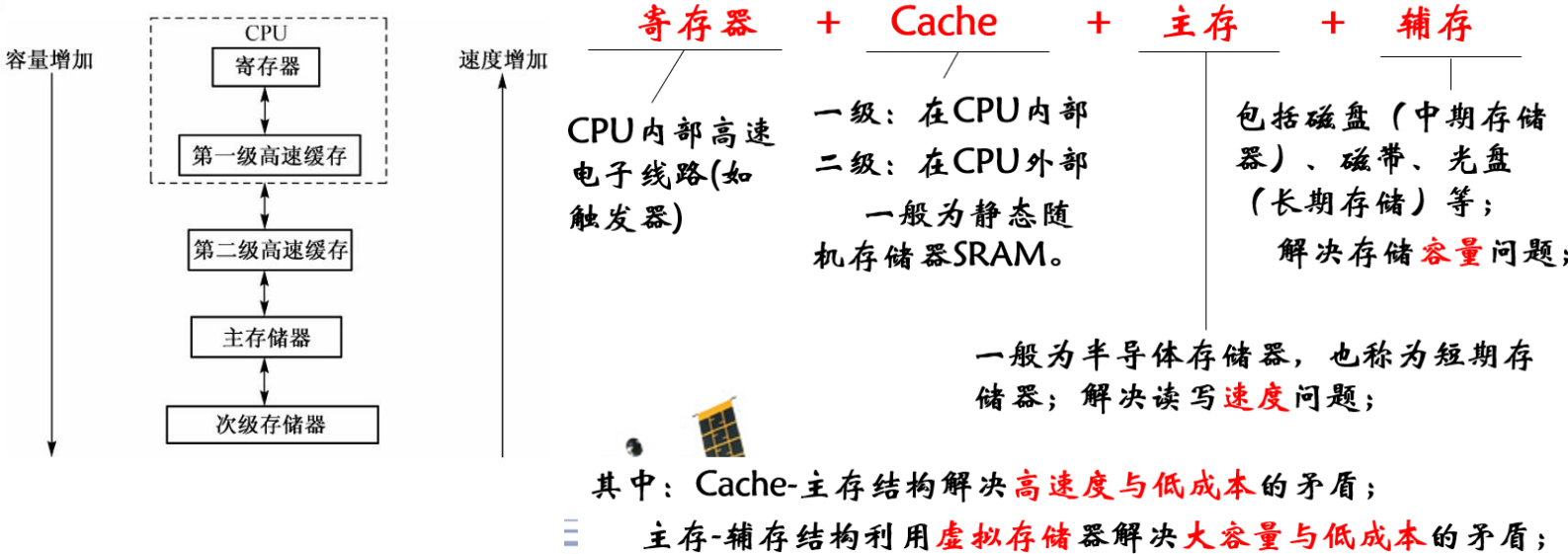
现假定外部设备有新的数据到来并发生了中断，CPU 在设定完 DMA 控制器后继续其工作，DMA 根据设定将 1M 的新数据装入 RAM 中并通知 CPU 新的数据到了。(此时，出现数据不一致)

解决的办法：在 CPU 访问异步事件控制的数据前一定要强行刷新 Cache 中的内容；——刷新

反之，从内存到外部设备搬移数据前一定要回写内存。——回写

一般 CPU 都提供了 Cache 的刷新和回写机制，甚至有的 CPU 还有 **Cache 保护**（即强制其不要对某一范围内的内存使用 Cache 机制）。

典型计算机系统中的存储器层四级存储结构

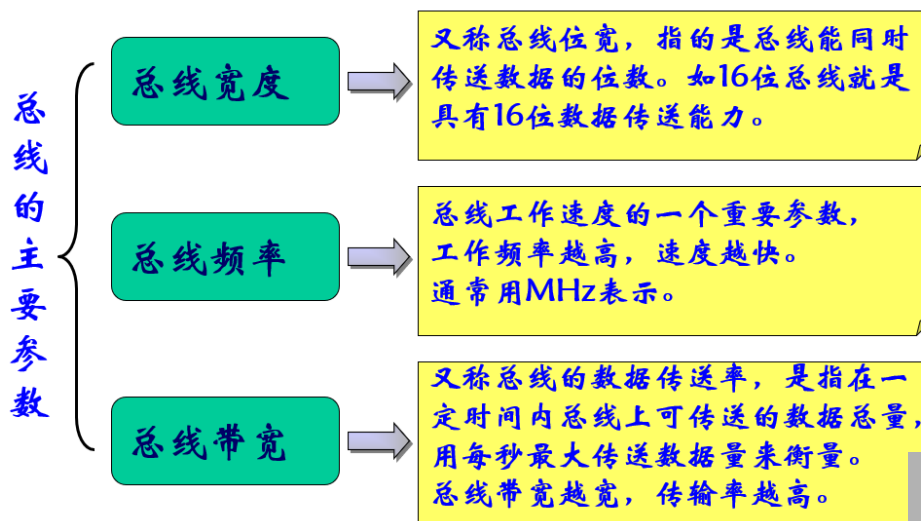
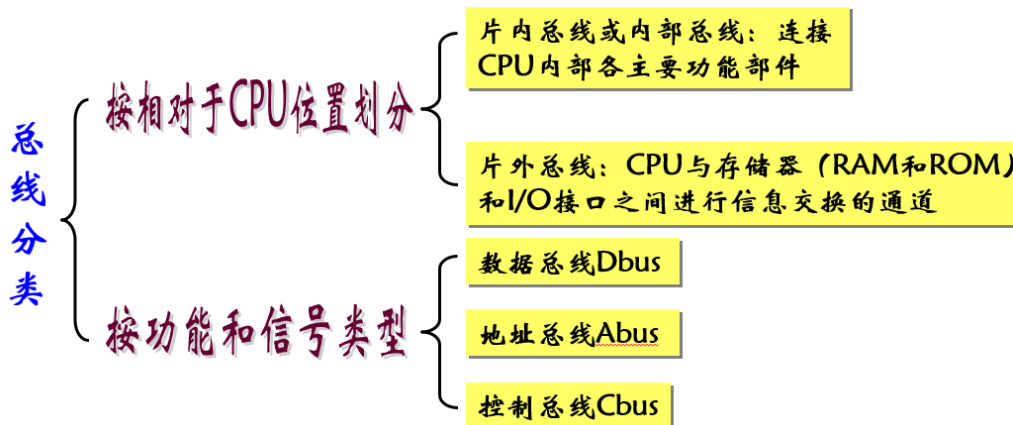


总线及总线桥

- **总线**是计算机中各模块传输信息的公共信号线集合。为各模块间，甚至模块各部件之间提供公共的、标准化的信息通路。
- **总线的特点**在于其**公用性**(即它可以同时挂接多个模块或设备，作为所有挂接模块或设备公用的信号载体)。总线在**同一时刻**，只能允许**一对模块或设备进行信息交换**。当有多个模块或设备需要同时使用总线进行信息传输时，只能采用**分时方式**，并且要对总线使用的**优先权进行仲裁管理**。

- 总线结构包括：传输信息的传输线路和总线控制逻辑
- 嵌入式系统采用总线结构有以下优点：
 - (1) 简化了系统结构。
 - (2) 优化了硬件和软件设计。
 - (3) 便于系统的扩充和更新。

总线是 CPU 与存储器和设备通信的机制，是计算机各部件之间传送数据、地址和控制信息的公共通道。



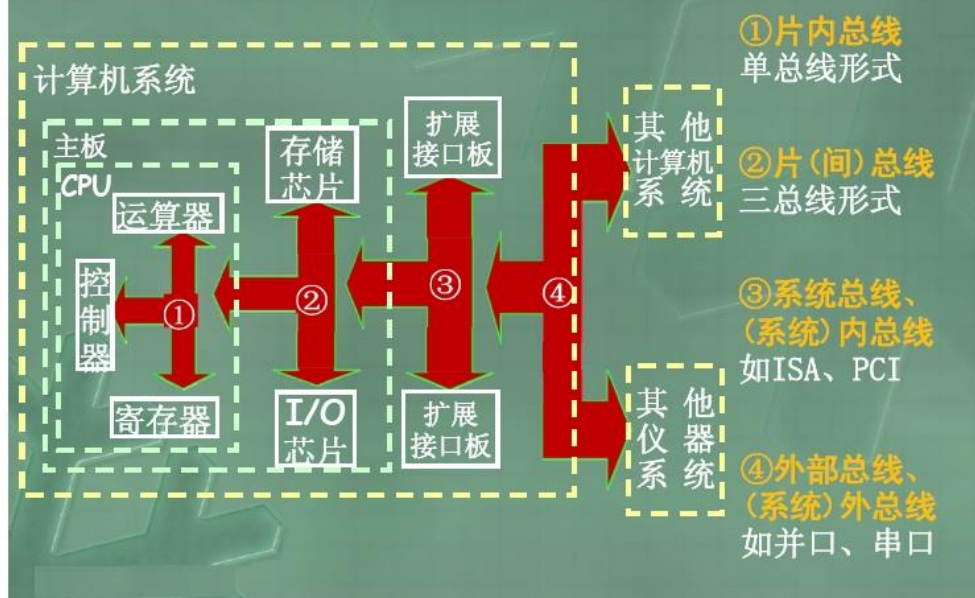
总线带宽(单位:MB/s)

$$= (\text{总线宽度}/8) \times \text{总线频率}$$

如：总线宽度32位，频率66MHz，则
总线带宽 = (32/8) * 66MHz = 264MB/s



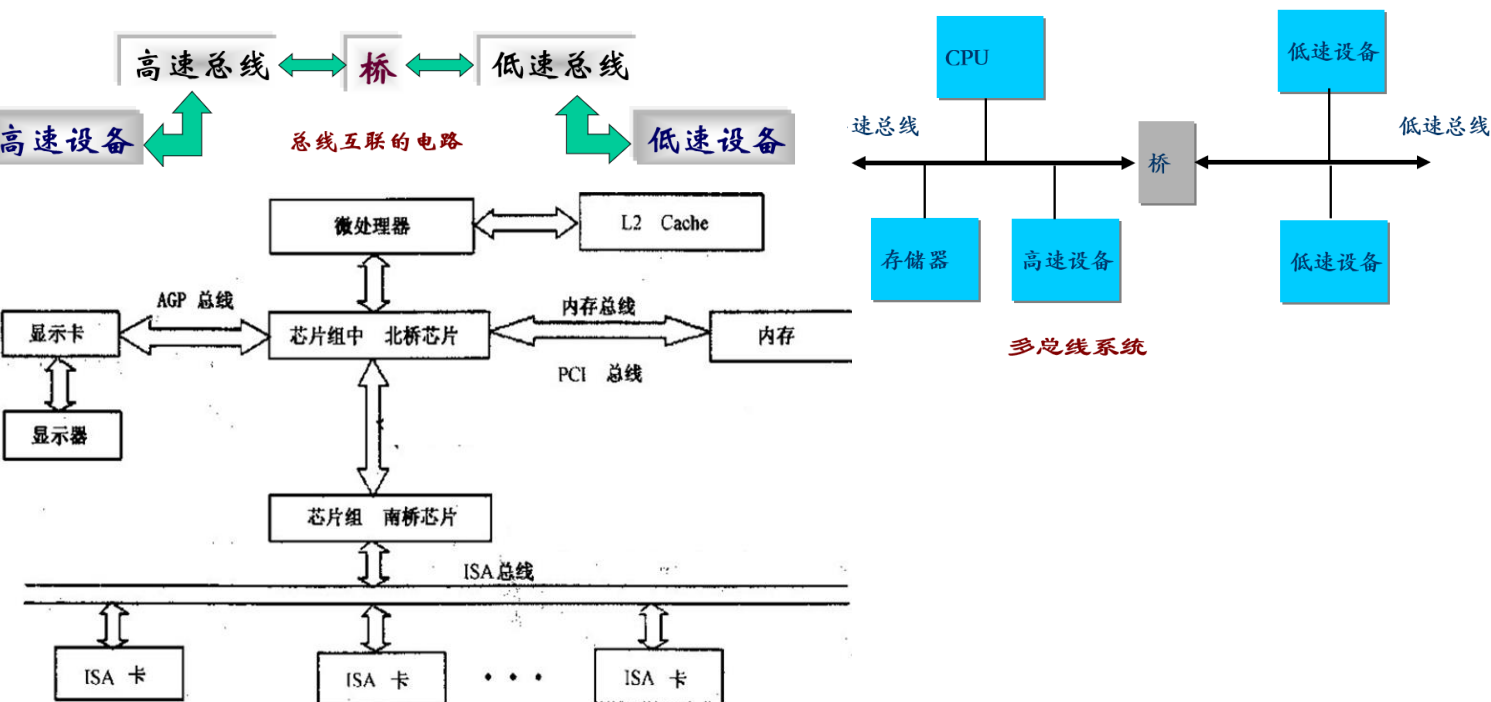
计算机系统的四层总线结构



一个微处理器系统可能含有多条总线

原因：

- 数据宽度：高速总线通常提供较宽的数据连接。
- 成本：高速总线通常采用更昂贵的电路和连接器。
- 桥允许总线独立操作，这样在 I/O 操作中可提供某些并行性。



典型微机系统中的总线层次

AMBA 总线

AMBA (Advanced Microcontroller Bus Architecture) 是 ARM 公司研发的一种总线规范，目前主要版本为 2.0 版本。

AHB(Advanced High-performance Bus)：用于高性能系统模块的连接，支持突发模式数据传输和事务分割；可以有效地连接处理器、片上和片外存储器，支持流水线操作。

APB (Advanced Peripheral Bus)：用于较低性能外设的简单连接，一般是接在 AHB 系统总线上的第二级总线。

第三讲 嵌入式系统硬件设计

一、 嵌入式系统硬件开发流程

项目需求、计划阶段

开始于项目需求分析，结束于总体技术方案确定。主要进行硬件设计需求分解，包括硬件功能需求、性能指标、可靠性指标、可制造性需求、可服务性需求及可测试性等需求；对硬件需求进行量化，并对其可行性、合理性、可靠性等进行评估，硬件设计需求是硬件工程师总体技术方案设计的基础和依据。

原型阶段

输入为总体技术方案，直到完成硬件概要设计为止。主要对硬件单元电路、局部电路或有新技术、新器件应用的电路的设计与验证及关键工艺、结构装配等不确定技术的验证及调测，为概要设计提供设计依据和设计支持。

开发阶段

开始于硬件概要设计评审通过后，结束于初样成功转为试样。主要有原理图及详细设计、PCB 设计、初样研制/加工及调测，每一阶段都要进行严格、有效的技术评审，以保证“产品的正确”。

验证阶段

各要素进行验证、优化的阶段，为大批量投产做最后的准备，开始于初样评审通过，结束于试样成功转产。主要有试样生产及优化改进、试样样机评审、转产；验证、改进过程要及时、同步修订、受控设计文档、图纸、料单等。

维护阶段

维护阶段开始于产品成功转产后，结束于产品生命周期结束。

嵌入式系统硬件

简要开发流程

电路规划：

- 根据硬件分配需求的要求，对器件资源进行规划，并编写“硬件规格书”。
 - 确定微处理器的接口资源是否满足需要
 - 确定微处理器对 ROM、RAM 容量和速度的支持是否满足需要
 - 微处理器通用输入输出接口分配
 - 中断分配
 - 微处理器外围芯片规划

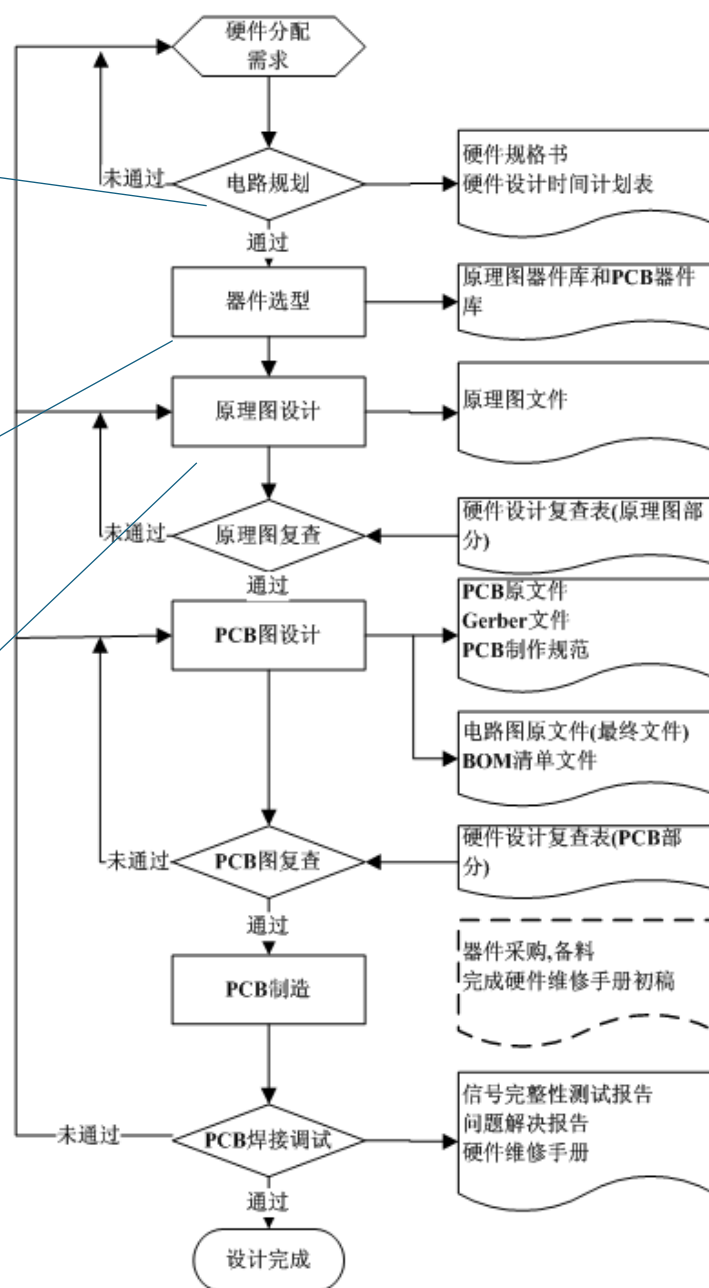
器件选型：

- 根据该系统对**功能和性能**的需求，寻找符合功能的器件。一般每个公司都会有自己的器件库，如果所用器件已经存在于数据库中，则可以直接选取。如果没有，则可以通过咨询代理商、经销商、互联网查询等方式来选择需要的器件。
- 在这个过程中，首先要选择的是**硬件系统的核心：嵌入式微处理器**，之后再根据选定的处理器的情况来选择其他器件

原理图设计：

- 原理图的设计原则主要遵从**先整体、后局部**的原则，先确定需要分成哪些模块，然后再进行细化。这样不但能使系统结构清晰，独立的模块也可以为以后的设计所使用，从而提高工作效率。
- 原理图设计—常用工具
 - ✓ PADS logic
 - ✓ Dxdesigner
 - ✓ Orcad capture
 - ✓ Protel99

原理图复查：



- ✓ **设计正确性。**检查各器件的连线是否正确，外围器件是否有遗漏，模块之间的连接关系是否正确，器件之间的网络名是否一致
- ✓ **设计完整性。**检查需要的功能是否都已实现
- ✓ **设计的可靠性。**检查选择器件的参数是否满足实际要求。
- ✓ **电磁兼容的考虑。**检查是否有针对电磁兼容增加或预留部分器件，以帮助解决电磁兼容问题

PCB 图设计：

- PCB 的设计需要遵从**先复杂后简单**的原则进行设计。先将设计中最复杂的部分或者难度最大的部分先行设计，这样可以**保证布局、走线最合理，性能最优化**。例如一个设计中的 CPU 部分（包括 CPU，外部存储器等）就应该优先进行设计。
- PCB 图设计—常用工具
 - ✓ PADS layout
 - ✓ Orcad Layout plus
 - ✓ Protel99
- **PCB 图设计步骤**
 - ✓ 确定 PCB 机构**尺寸**
 - ✓ 从原理图设计软件生成网络表并导入 PCB 设计软件
 - ✓ **器件布局**：采用先核心器件，后外围器件的原则进行
 - ✓ **布线**：对于核心器件和高速信号要优先进行布线
 - ✓ **后处理**：主要包括修改不合理的走线或者多余的走线；电源层平面分割；铺铜；排列文字；添加各种表示符号等
- **PCB 图复查**
 - ✓ **布局**：比如器件之间的间距是否过小或过大，器件距离板边是否过近，芯片和其他器件的布局是否利于焊接维护，接口器件的方向是否正确，相关器件是否放置较为靠近等
 - ✓ **走线**：走线是否有直角，走线上是否有多余的过孔或不必要的转角，走线是否根据电流的不同而更改粗细，走线分叉部分是否尽量短，多颗芯片共用走线是否尽量以菊花链的方式连接等

PCB 制造：

- PCB 图经过检查无误后，生成 gerber 文件，然后将此文件发送给 PCB 厂商进行 PCB 制作。
- 生成 gerber 文件的作用一是可以对原设计文件进行保密，二是在该文件中各项设计参数的标注是明确的，避免引起 PCB 厂商对设计的误解

PCB 焊接及调试：

- PCB 制作完成后，硬件工程师需要对 PCB 板进行初步的检查，主要是**检查电源**是否存在短路问题，特别是对设计后第一次制作的 PCB 板更应注意。
- 接下来首先**焊接电源**部分并检查电压是否正常，如没有问题则可以进行其他器件的焊接。
- 焊接完成后需要再次进行检查，主要是看系统的各种电压是否与设计值一致，时钟是否正常，复位信号是否正常。
- 编写简单的**硬件驱动软件**，也可以对硬件是否正常工作进行测试，此时需要对整个系统的信号完整性进行分析测量，确保各个器件的实际工作电压、电流值在设计规定的范围之内，器件的工作时序满足要求等

嵌入式系统的设计原则

- 要有**明确的需求**，采用自上而下的方法进行设计
- 采用**模块化设计**方法，对功能模块进行划分
- **选择器件**需要从性能、可靠性、成本等方面进行考虑

- **可靠性设计**可根据需要，采用器件参数降额使用或者采用冗余设计等方法来提高和保证

二、 嵌入式系统硬件设计分析

案例分析——定位系统：

硬件分配需求

定位：

- **主动定位**：待定位设备主动发出定位信号，用于定位系统对该设备进行侦测定位。
定位信号选择： 光波 / 声波 / 电磁波
定位系统构建： 无线传感网络 / 蜂窝网络
- **被动定位**：待定位设备根据定位系统发出的定位信号进行自我定位。
定位信号选择： GPS / 北斗 / 特殊定位信号

通信：定位结果如何上报——GSM / 3G / 4G / 5G？

需求（方案）1：采用 GPS/北斗进行定位，利用 **GSM** 网络定期短信上报定位结果（北斗+GSM 模块+单片机）

特性需求：

定位设备上是否需要存储所有定位结果

定位设备是否支持定位结果导出功能

定位设备是否支持路线偏离预警

需求（方案）2：进行定位，定期上报定位结果；可以一次性导出所有存储的定位结果；可以配置预设路线或区域，进行定位偏离预警（北斗定位模块，GSM 模块，ARM，NAND/NOR Flash，串口）

人机交互：PC 机无串口接口，串口配置不方便；利用手机进行近距离通信，实现配置/导出

需求（方案）3：定位、上报；配置导入导出、偏离预警；近距离通信（北斗定位模块，GSM 模块，ARM，NAND/NOR Flash，蓝牙/WIFI）

特色需求：一键紧急告警；实时跟踪（定位周期小于 1 秒）；

需求（方案）4：定位、上报；配置导入导出、偏离预警；近距离通信；一键告警、实时监控（北斗定位模块，4G/5G，ARM，NAND/NOR Flash，蓝牙/WIFI）

特色需求：双向语音对讲；偏离预设路线或范围时震动提示、红灯提示

需求（方案）5：北斗定位模块，4G/5G，ARM，NAND/NOR Flash，语音采集、播放，振动马达，LED 指示灯（定位监控，电子围栏，紧急报警，双向语音，提示功能）

硬件设计总结：

北斗定位模块	外设
4G/5G	网络
ARM	CPU
NAND/NOR Flash	存储
语音采集、播放	I/O
振动马达	外设
LED 指示灯	I/O

三、 嵌入式系统在物联网中的应用

什么是物联网：

- 物联网（The Internet of things）：就是**物物相连的互联网**；基于互联网、传统电信网等信息承载体，让所有能够被独立寻址的普通物理对象实现互联互通的网络。
- 这有两层意思：
 - ✓ 第一，物联网的核心和基础仍然是互联网，是在互联网基础上的延伸和扩展的网络；

✓ 第二，其用户端延伸和扩展到了任何物品与物品之间，进行信息交换和通信。

物联网结构：

应用实现层（应用）

信息传输层

信息采集层

物联网应用：智能家居，农业，物流，交通，安防，电网，环保，医疗，工业

信息采集层：

信息传输层：

信息采集层

RFID射频类

身份识别

车辆识别

传感类

温度传感

湿度传感

烟雾传感

风速传感

光线传感

影像采集

摄像头

可视门禁

信息传输层

无线传输

WIFI

有线传输

互联网

局域网

手机

平板电脑

笔记本或者
台式机

应用实现层：

手机

运用java程序开发对应信息采集的软件开发，
实现智能家居的运用

平板

运用java、object-C程序开发对应信息采集
的软件开发，实现智能家居的运用

电脑

运用C++、C#程序开发对应信息采集的软件
开发，实现智能家居的运用